

bok25

基
礎

課、計算機網絡



数据链路层概述

- 概述
- 封装成帧
- 透明传输
- 差错检测

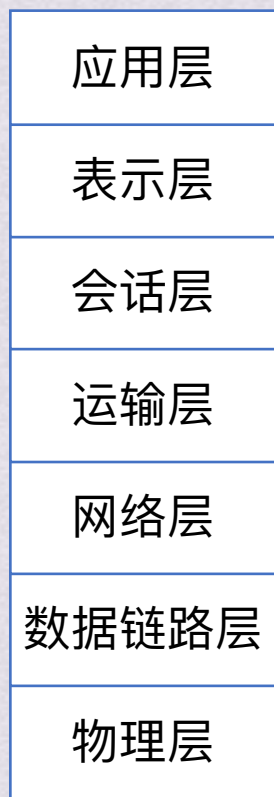


概述

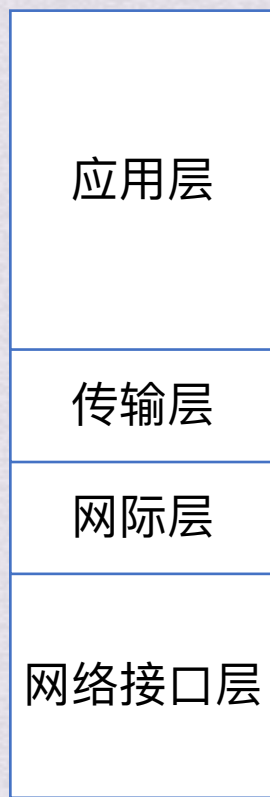




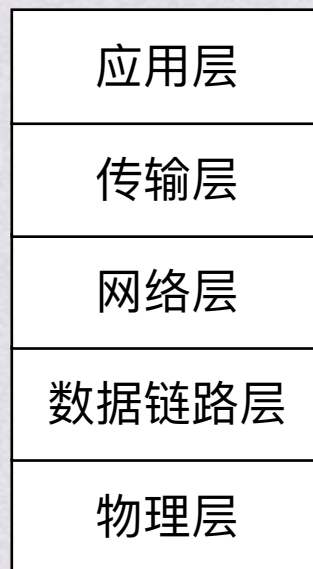
概述



OSI参考模型



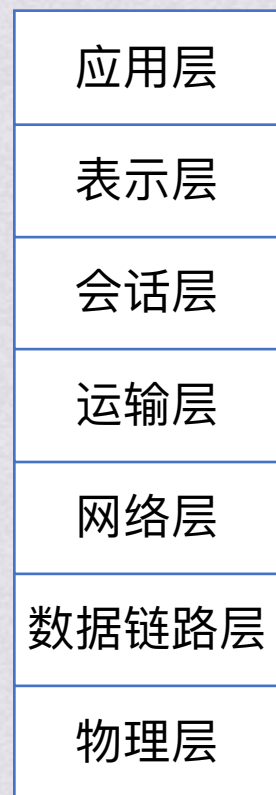
TCP/IP模型



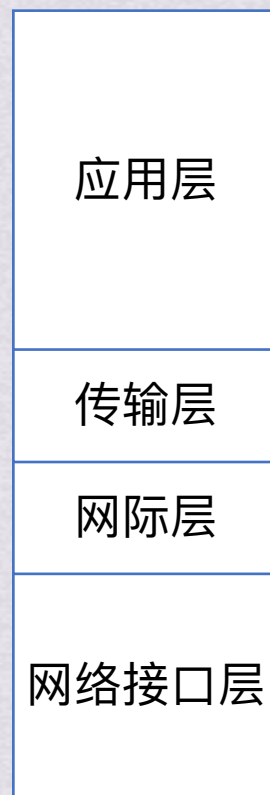
本门课程学习的层次



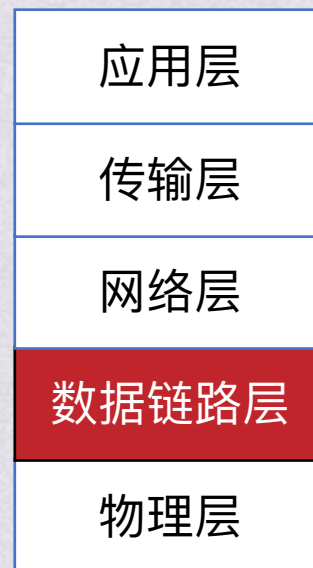
概述



OSI参考模型



TCP/IP模型



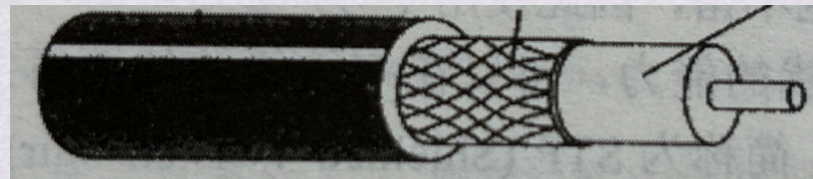
本门课程学习的层次



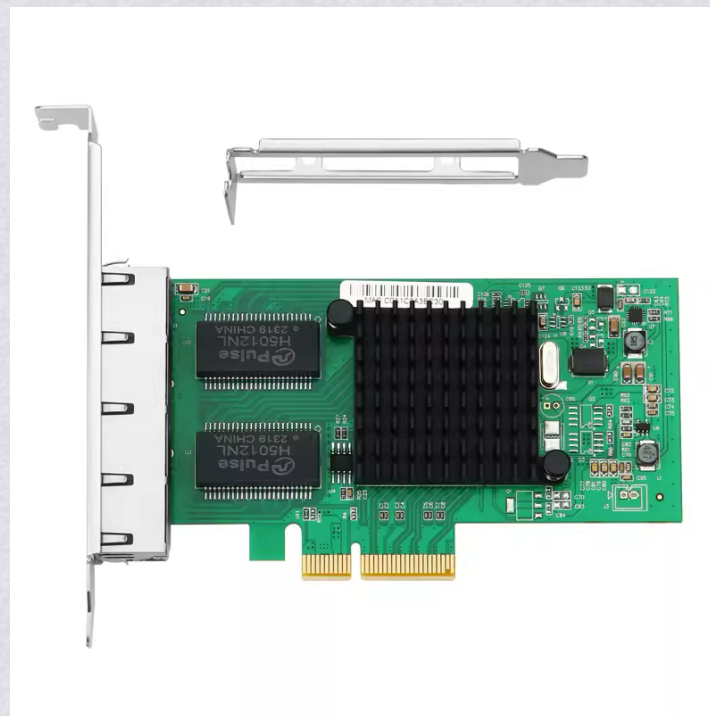
概述

数据链路层

≠



数据链路：物理链路和实现协议的软硬件的总和

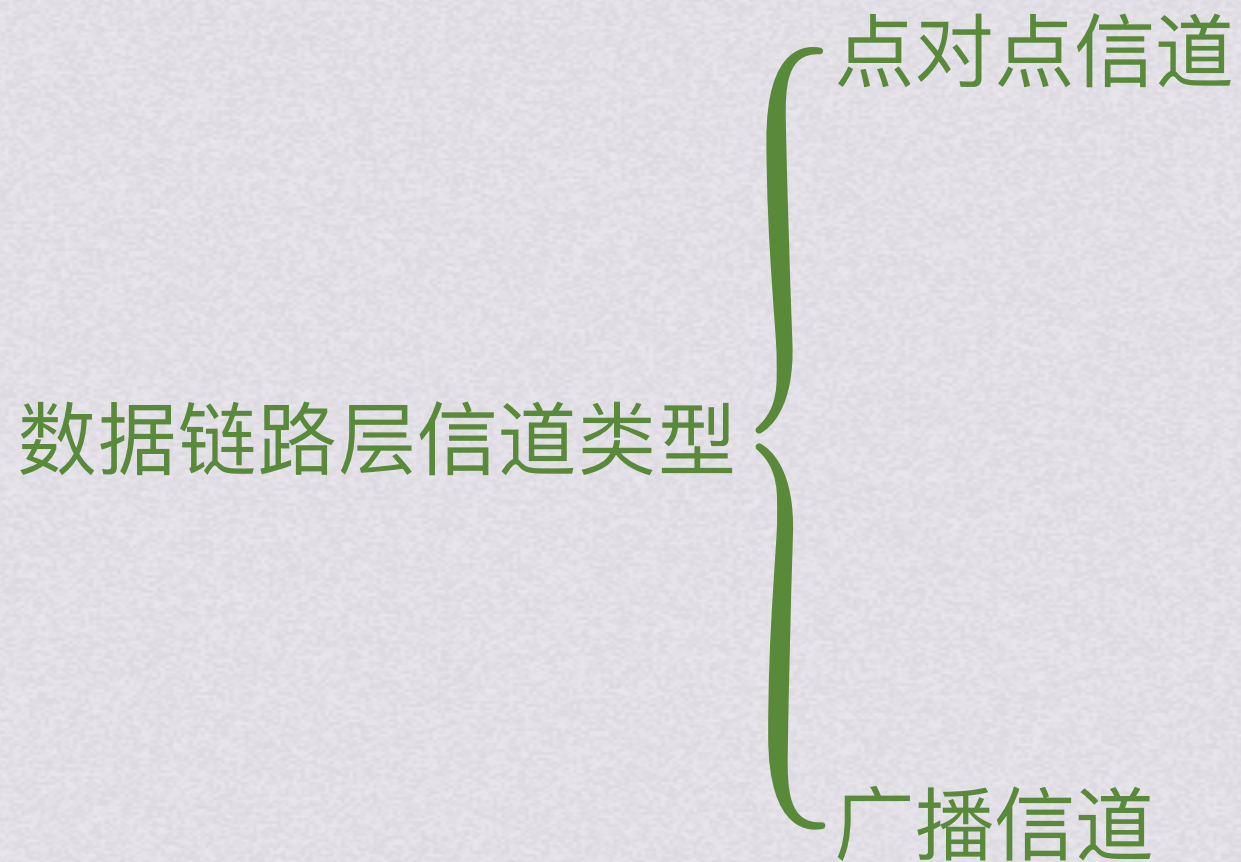


市面上常见的网卡(网络适配器)

更多的网卡直接集成在了主板上，作为主板的一部分
网卡工作在物理层和数据链路层



概述



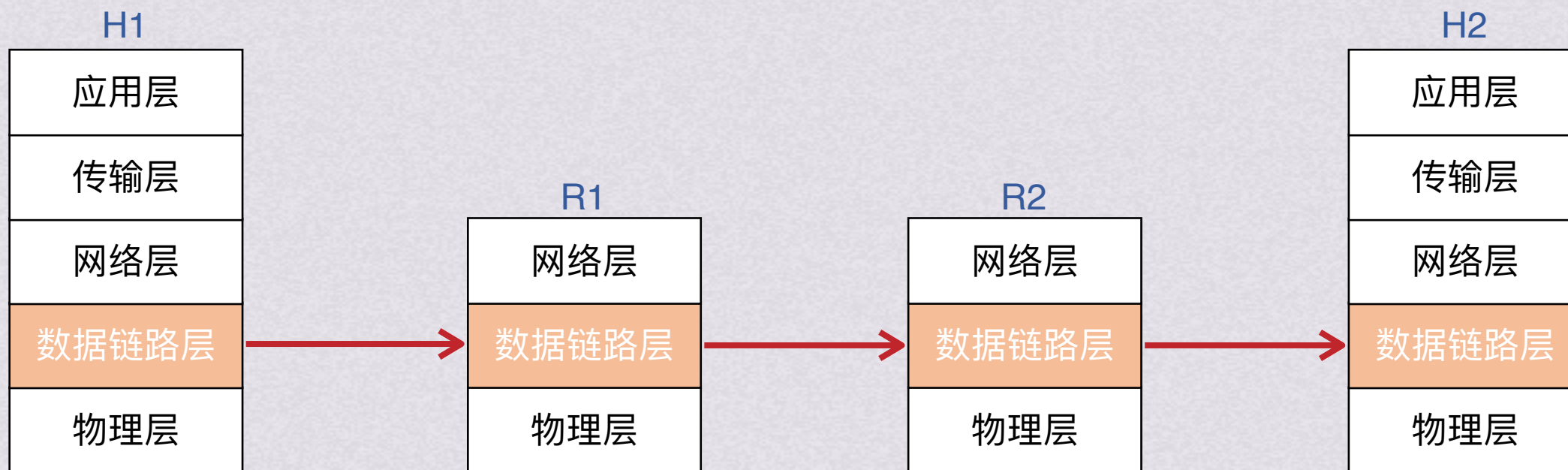


概述



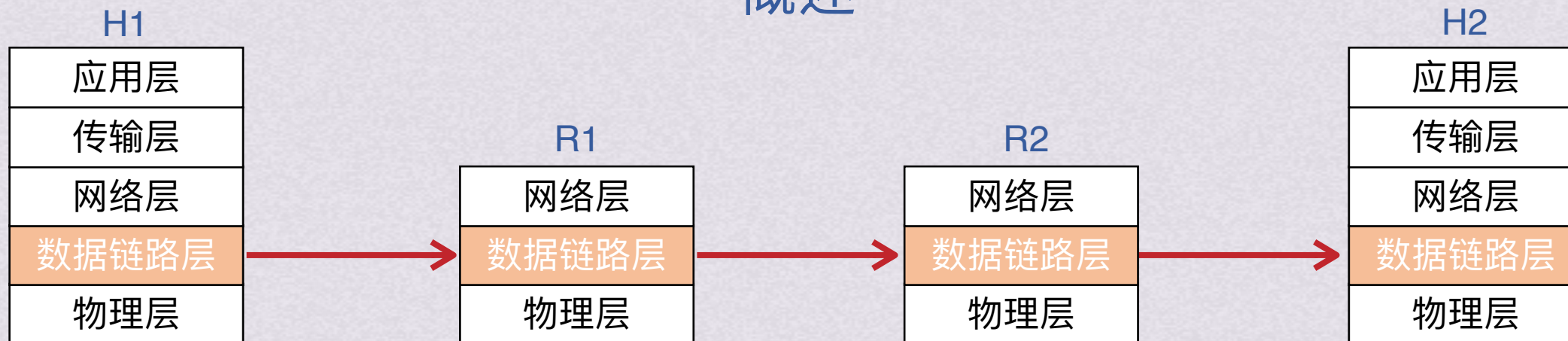


概述



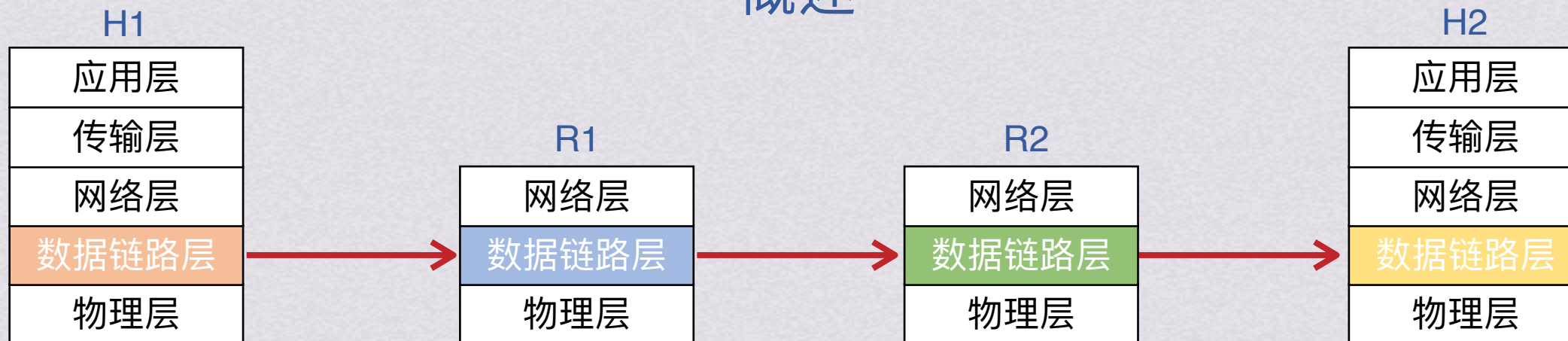


概述





概述

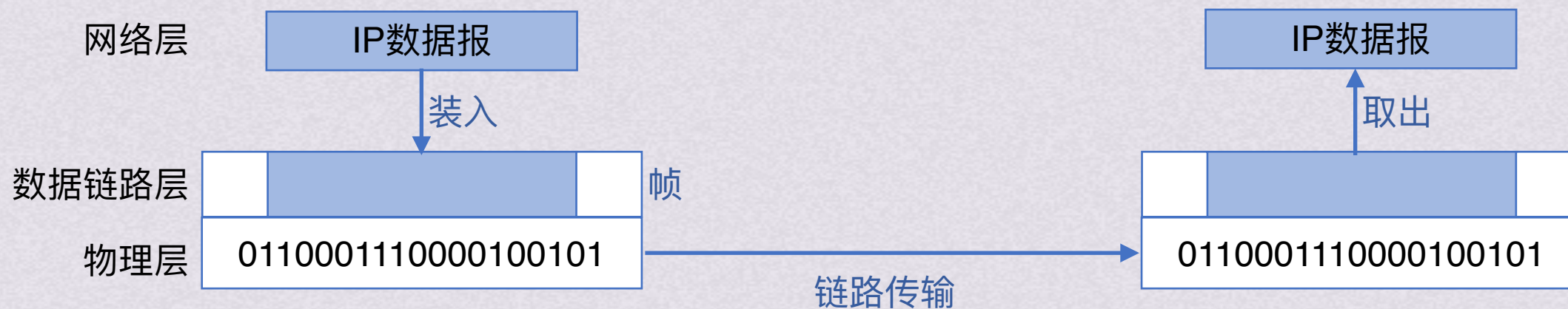
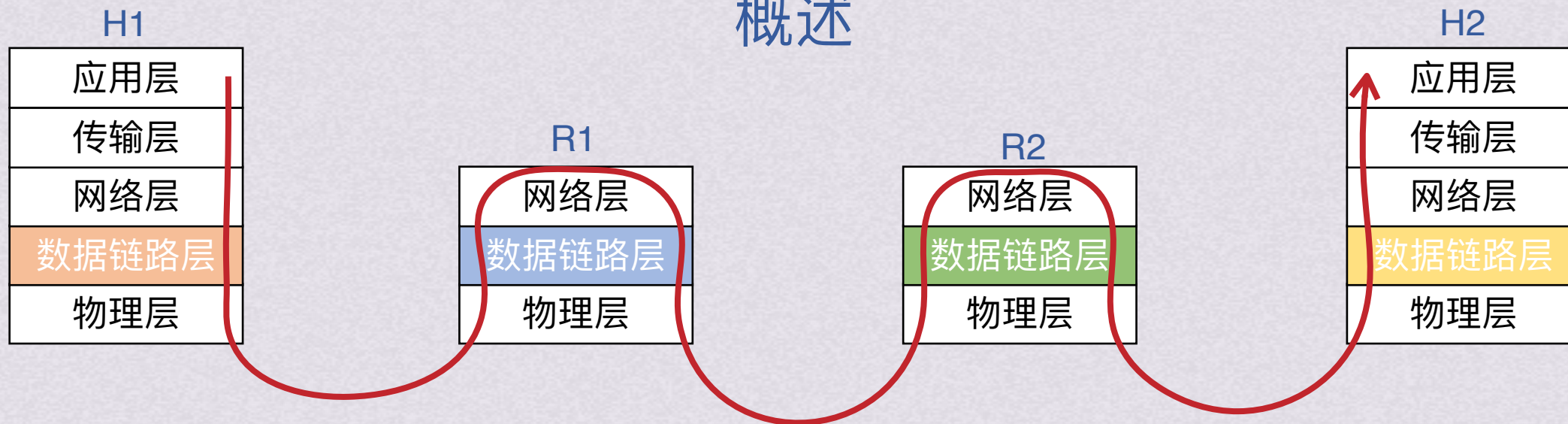


不同的链路层可能使用了不同的数据链路层协议



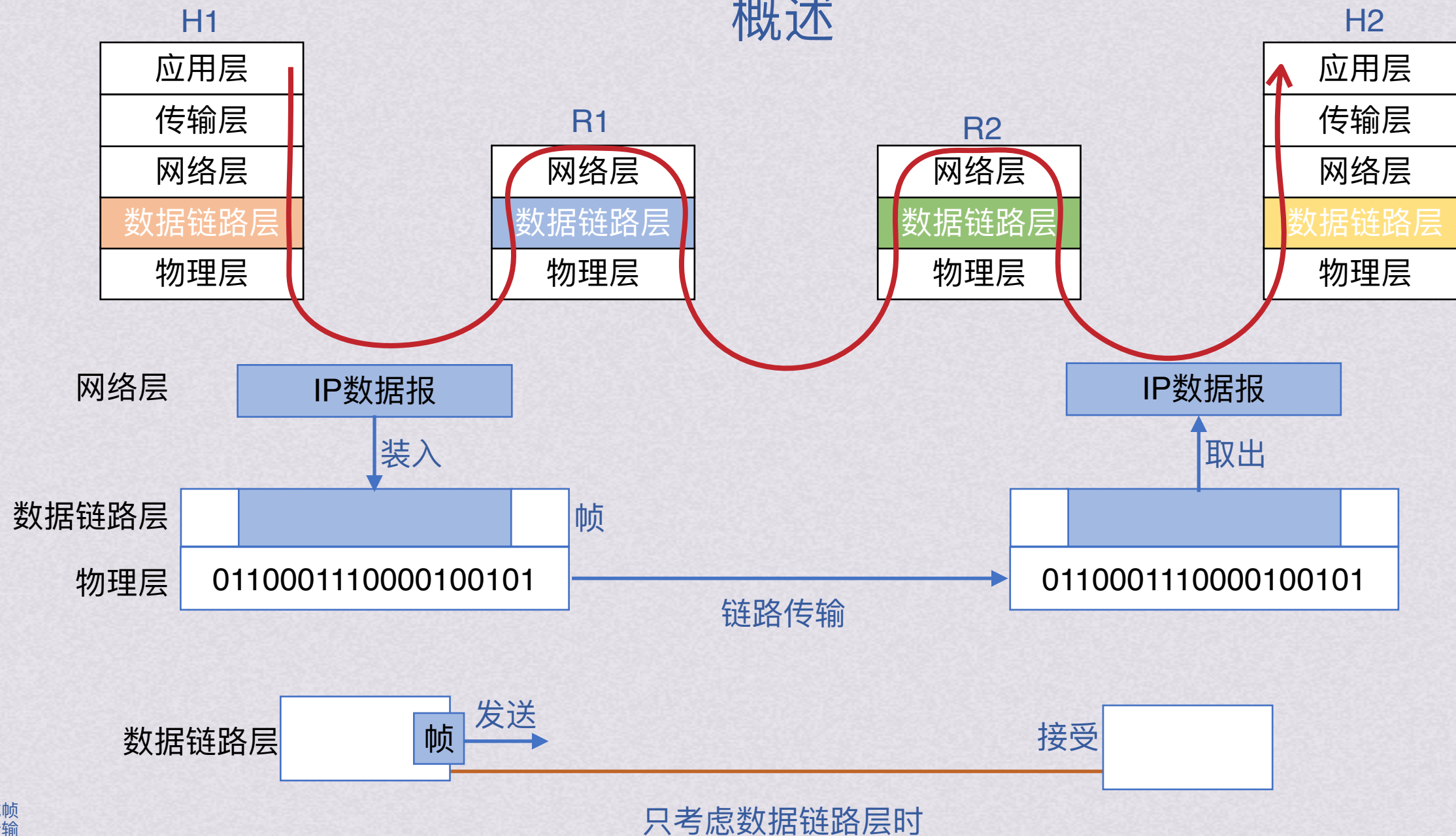


概述



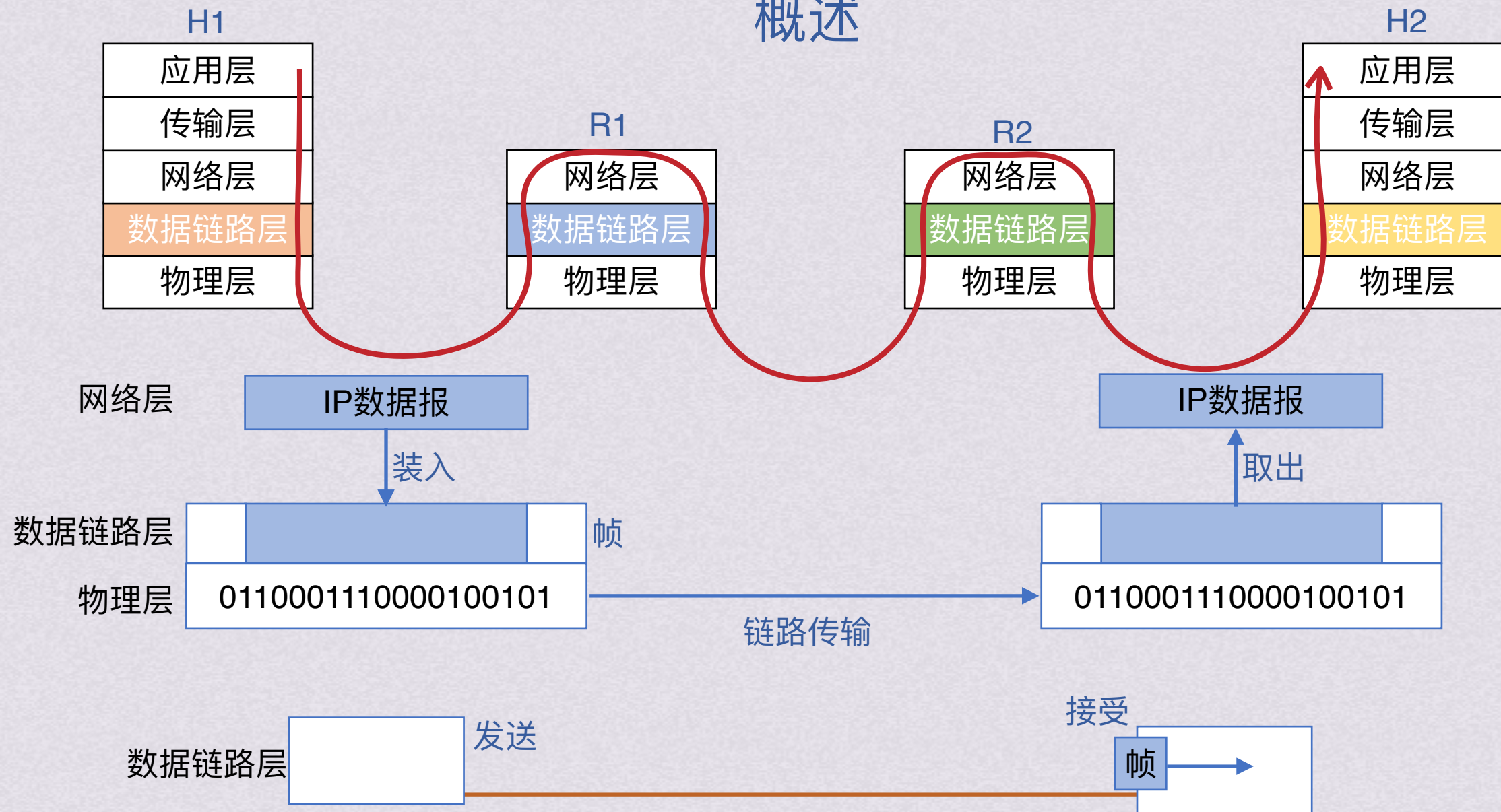


概述





概述



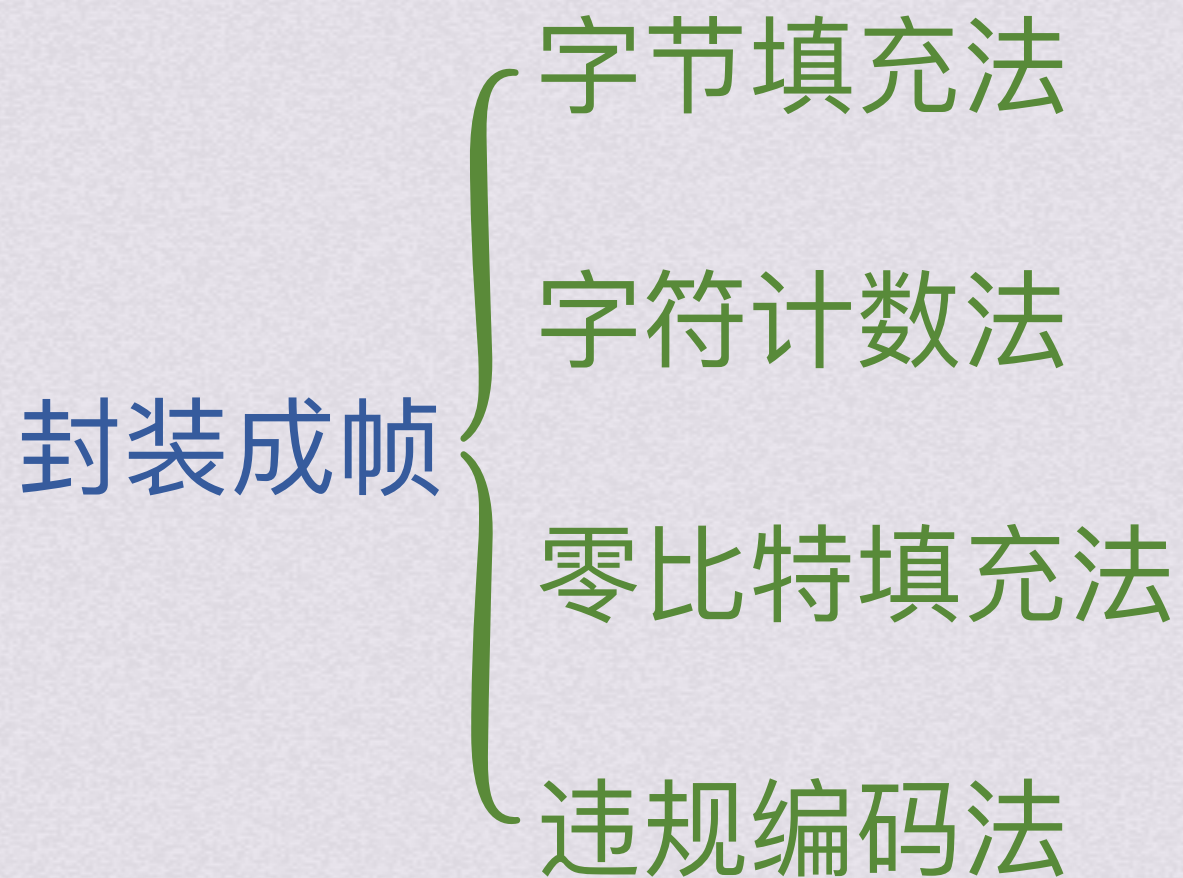
只考虑数据链路层时



概述



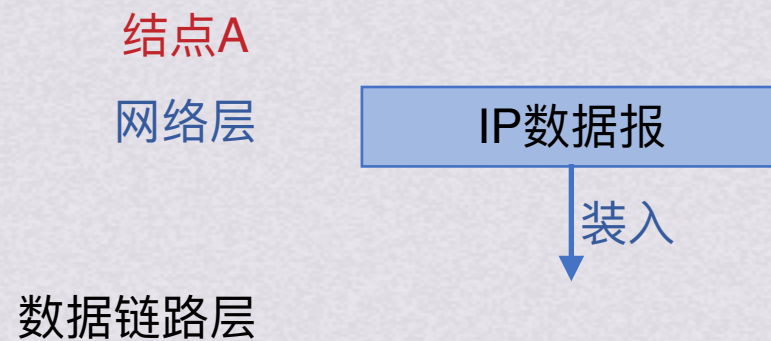
封装成帧





封装成帧

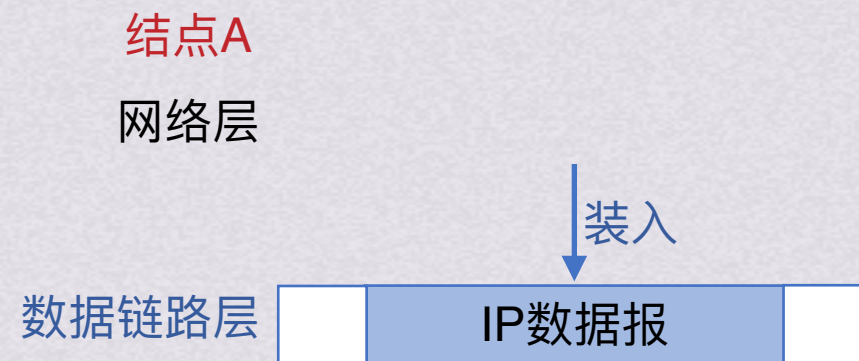
字节填充法





封装成帧

字节填充法





封装成帧

字节填充法





封装成帧

字节填充法

结点A

网络层

数据链路层

物理层

0110001110000100101



封装成帧

字节填充法

结点A

网络层

结点B

网络层

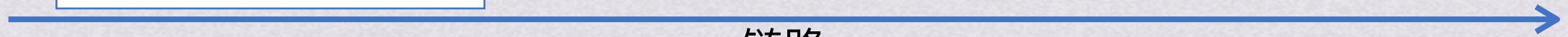
数据链路层

数据链路层

物理层

01100011110000100101

物理层

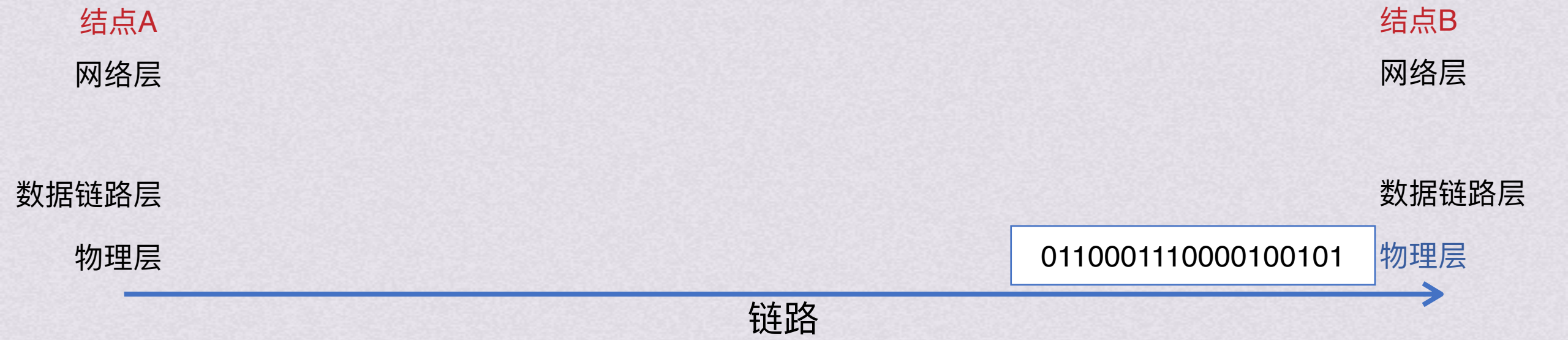


链路



封装成帧

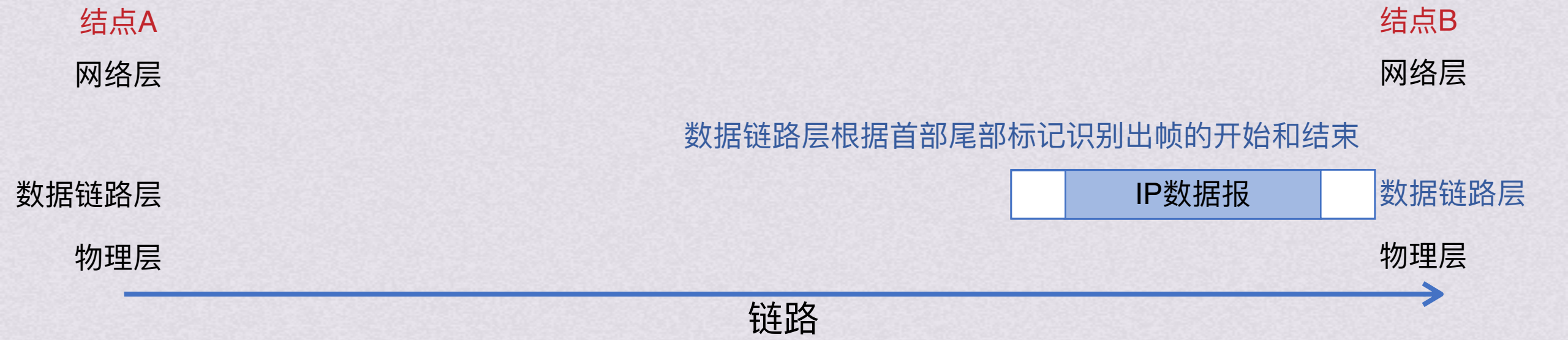
字节填充法





封装成帧

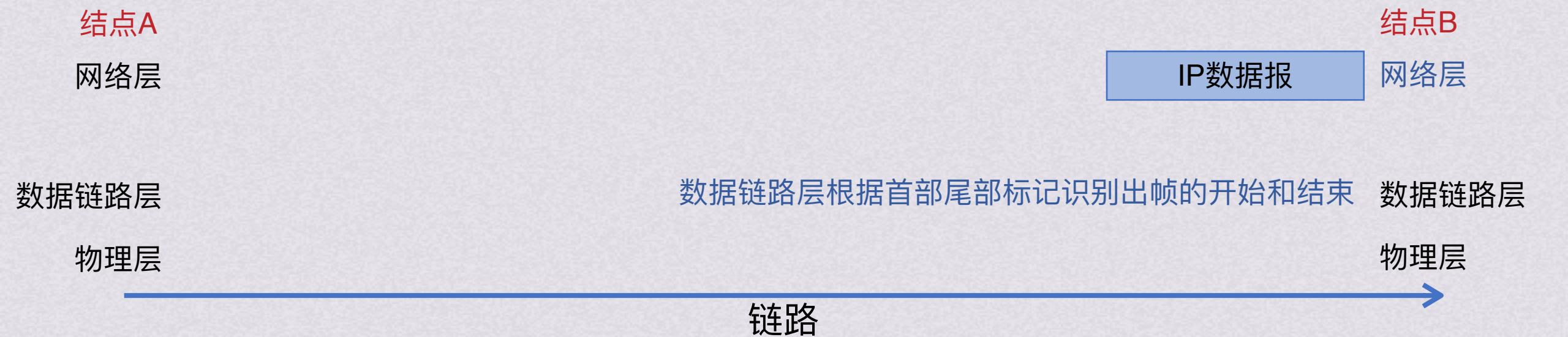
字节填充法





封装成帧

字节填充法





封装成帧

字节填充法



首部和尾部最重要的作用：帧定界
也就是从收到的比特流中识别出一个帧的开始和结束



封装成帧

字节填充法



首部和尾部最重要的作用：帧定界
也就是从收到的比特流中识别出一个帧的开始和结束

当数据是可打印的（即可以从键盘上输入的字符）ASCII码组成的文本时，帧定界可以使用特殊的帧定界符

SOH放在最前面，表示帧的首部开始

EOT放在一帧的最后面，表示帧的结束

SOH是二进制0000 0001

EOT是二进制0000 0100



封装成帧



当一个帧的首部或者尾部出现问题时，可以根据其他帧的首尾部知道收到的数据哪一部分需要丢弃

首部和尾部最重要的作用：帧定界
也就是从收到的比特流中识别出一个帧的开始和结束

当数据是可打印的（即可以从键盘上输入的字符）ASCII码组成的文本时，帧定界可以使用特殊的帧定界符

SOH放在最前面，表示帧的首部开始
EOT放在一帧的最后面，表示帧的结束

SOH是二进制0000 0001

EOT是二进制0000 0100



封装成帧



当一个帧的首部或者尾部出现问题时，可以根据其他帧的首尾部知道收到的数据哪一部分需要丢弃

首部和尾部最重要的作用：帧定界
 也就是从收到的比特流中识别出一个帧的开始和结束

当数据是可打印的（即可以从键盘上输入的字符）ASCII码组成的文本时，帧定界可以使用特殊的帧定界符

SOH放在最前面，表示帧的首部开始
 EOT放在一帧的最后面，表示帧的结束

SOH是二进制0000 0001

EOT是二进制0000 0100



封装成帧



当一个帧的首部或者尾部出现问题时，可以根据其他帧的首尾部知道收到的数据哪一部分需要丢弃

首部和尾部最重要的作用：帧定界

也就是从收到的比特流中识别出一个帧的开始和结束

当数据是可打印的（即可以从键盘上输入的字符）ASCII码组成的文本时，帧定界可以使用特殊的帧定界符

SOH放在最前面，表示帧的首部开始

EOT放在一帧的最后面，表示帧的结束

SOH是二进制0000 0001

EOT是二进制0000 0100



封装成帧

字符计数法

在帧首部用一个计数字段来记录该帧所含的字节数（包括计数字段自身占用的1个字节）



但这种方法一旦计数字段出错，失去了帧边界划分的依据，收发双方会失去同步，造成灾难性后果



封装成帧

零比特填充法

在HDLC协议中使用这种方法，允许数据帧包含任意个数的比特，使用0111 1110来表示帧的开始和结束

0	1	1	1	1	1	0	0	1	1	1	1	1	1	0	1	1	0	0	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

原始数据

数据中如果出现了连续的5个1，就填充一个0，这样就能保证数据中不会出现连续的6个1

0	1	1	1	1	1	1	0	0	1	1	1	1	1	0	0	0	1	1	1	1	1	0	1	0	1	1	0	0	1	0	1	1	1	1	1	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

传送数据

接收方再做相反的操作，每收到5个连续的1，就自动删除后面跟着的0

零比特填充法很容易用硬件实现，性能优于字节填充法

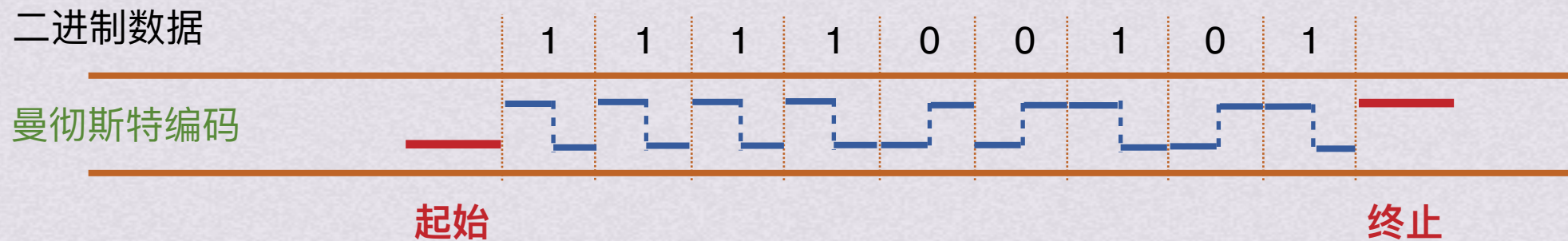
是数据链路层早期使用的HDLC协议中用来实现透明传输的办法



封装成帧

违规编码法

物理层进行比特编码时常采用的方法，采用一些在数据比特中原本是违规的，没有采用的方法来定界帧的起始和终止



局域网IEEE802标准就采用了这种方法，违规编码法不采用任何填充技术就能实现透明传输，但只适用于采用冗余编码的特殊编码环境

因为字符计数法中计数字段的脆弱性和字节填充法实现上的复杂和不兼容性，目前常用的组帧方法是零比特填充法和违规编码法



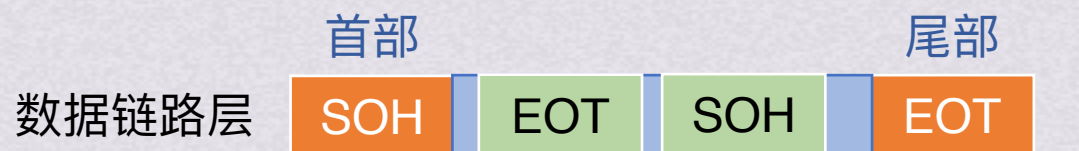
封装成帧



透明传输



透明传输



SOH放在最前面，表示帧的首部开始

EOT放在一帧的最后面，表示帧的结束

SOH是二进制0000 0001

EOT是二进制0000 0100

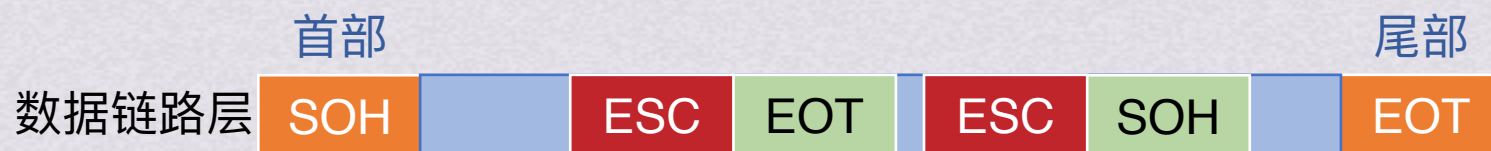
如果要传输的数据中恰好含有SOH或者EOT的数据，那还能够正确传输吗？

为此引入转义字符ESC，十六进制编码1B，二进制0001 1011

在数据中出现SOH或EOT时，在前面插入一个ESC即可



透明传输



SOH放在最前面，表示帧的首部开始

EOT放在一帧的最后面，表示帧的结束

SOH是二进制0000 0001

EOT是二进制0000 0100

如果要传输的数据中恰好含有SOH或者EOT的数据，那还能够正确传输吗？

为此引入转义字符ESC，十六进制编码1B，二进制0001 1011

在数据中出现SOH或EOT时，在前面插入一个ESC即可



透明传输



SOH放在最前面，表示帧的首部开始

EOT放在一帧的最后面，表示帧的结束

SOH是二进制0000 0001

EOT是二进制0000 0100

如果要传输的数据中恰好含有SOH或者EOT的数据，那还能够正确传输吗？

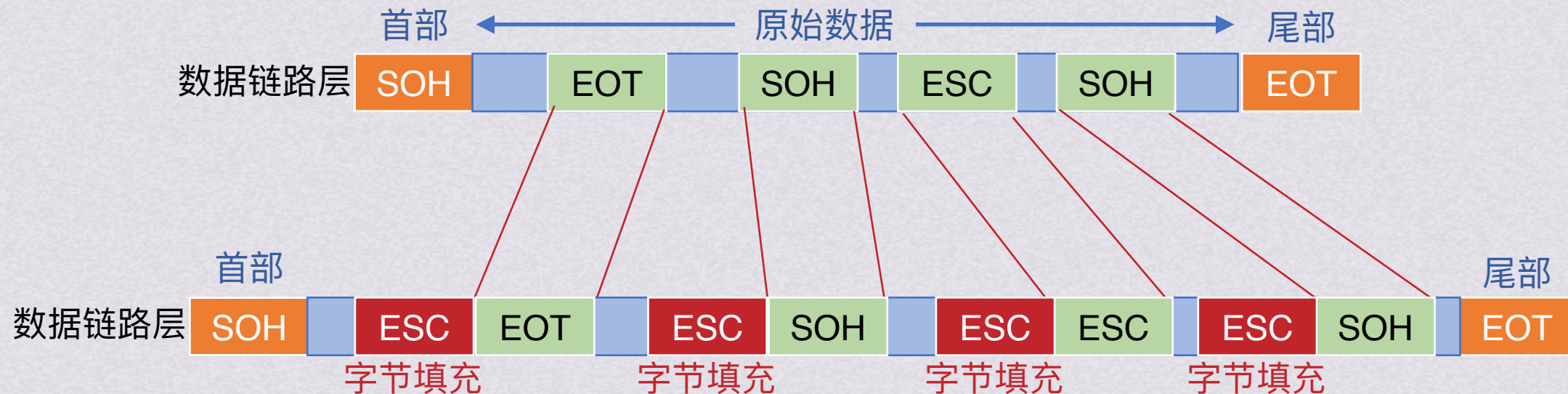
为此引入转义字符ESC，十六进制编码1B，二进制0001 1011

在数据中出现SOH或EOT时，在前面插入一个ESC即可

如果在数据中再出现了转义字符，那就在转义字符前面再加一个转义字符



透明传输



如果要传输的数据中恰好含有SOH或者EOT的数据，那还能够正确传输吗？

为此引入转义字符ESC，十六进制编码1B，二进制0001 1011

在数据中出现SOH或EOT时，在前面插入一个ESC即可

如果在数据中再出现了转义字符，那就在转义字符前面再加一个转义字符



透明传输



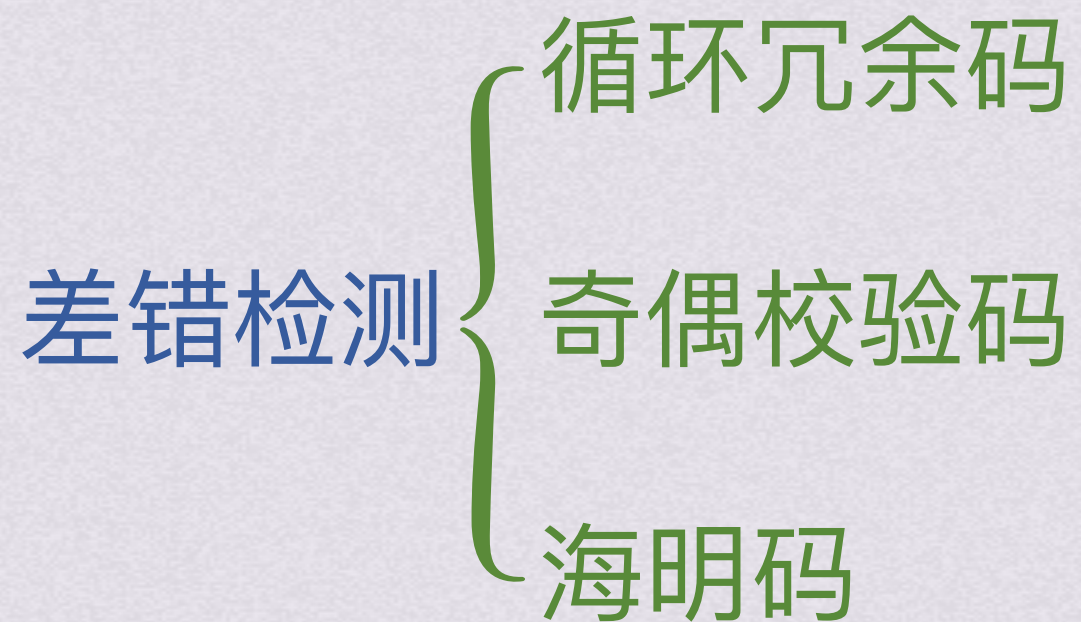
差错检测



差错检测



比特在传输过程中可能会发生差错，1可能变成0,0可能变成1，这就是比特差错，本章的差错检测部分只讨论比特差错





差错检测

循环冗余码

循环冗余码的使用步骤如下：

- 1.收发双方约定好一个**生成多项式 $G(x)$**
- 2.发送方根据要发送的数据和生成多项式计算出差错检测码(冗余码)，将其添加到待传输数据的后面一起传输
- 3.接收方通过生成多项式来计算收到的数据是否产生了误码

例：假设要发送的数据为101001，生成多项式为 $G(x)=x^3 + x^2 + 1$,如何计算出要发送的数据？

$$G(x)=x^3 + x^2 + 1 = \underset{\triangle}{1} \times x^3 + \underset{\triangle}{1} \times x^2 + \underset{\triangle}{0} \times x^1 + \underset{\triangle}{1} \times x^0 \rightarrow \text{生成多项式构成的比特串为1101}$$

按照多项式的最高有效次项给数据补0



差错检测

循环冗余码

循环冗余码的使用步骤如下：

- 1.收发双方约定好一个**生成多项式 $G(x)$**
- 2.发送方根据要发送的数据和生成多项式计算出**差错检测码(冗余码)**，将其添加到待传输数据的后面一起传输
- 3.接收方通过生成多项式来计算收到的数据是否产生了误码

例：假设要发送的数据为101001，生成多项式为 $G(x)=x^3 + x^2 + 1$ ，如何计算出要发送的数据？

$$G(x)=x^3 + x^2 + 1 = \underset{\triangle}{1} \times \boxed{x^3} + \underset{\triangle}{1} \times x^2 + \underset{\triangle}{0} \times x^1 + \underset{\triangle}{1} \times x^0 \rightarrow \text{生成多项式构成的比特串为 } 1101$$

按照多项式的最高有效次项给数据补0

1 0 1 0 0 1 0 0 0



差错检测

循环冗余码

循环冗余码的使用步骤如下：

- 1.收发双方约定好一个**生成多项式G(x)**
- 2.发送方根据要发送的数据和生成多项式计算出**差错检测码(冗余码)**，将其添加到待传输数据的后面一起传输
- 3.接收方通过生成多项式来计算收到的数据是否产生了**误码**

例：假设要发送的数据为101001，生成多项式为 $G(x)=x^3 + x^2 + 1$ ，如何计算出要发送的数据？

$$G(x)=x^3 + x^2 + 1 = \underset{\triangle}{1} \times \boxed{x^3} + \underset{\triangle}{1} \times x^2 + \underset{\triangle}{0} \times x^1 + \underset{\triangle}{1} \times x^0 \rightarrow \text{生成多项式构成的比特串为1101}$$

按照多项式的最高有效次项给数据补0
之后用模2除法，将补0后的数据与生成多项式的比特串相除

1101

+

101001000

1101

1110

1101

1110

1101

1100

1101

1

110101

只要比特数量足够就上1，不用管够不够减

因为就根本不是减法

异或运算，不同就是1，相同就是0

最后把剩下的余数加到被除数即可得到发送出去的数据



差错检测

循环冗余码

循环冗余码的使用步骤如下：

- 1.收发双方约定好一个**生成多项式 $G(x)$**
- 2.发送方根据要发送的数据和生成多项式计算出**差错检测码(冗余码)**，将其添加到待传输数据的后面一起传输
- 3.接收方通过生成多项式来计算收到的数据是否产生了**误码**

例：假设要发送的数据为101001，生成多项式为 $G(x)=x^3 + x^2 + 1$ ，如何计算出要发送的数据？

$$G(x)=x^3 + x^2 + 1 = \underset{\triangle}{1} \times \boxed{x^3} + \underset{\triangle}{1} \times x^2 + \underset{\triangle}{0} \times x^1 + \underset{\triangle}{1} \times x^0 \rightarrow \text{生成多项式构成的比特串为 } 1101$$

按照多项式的最高有效次项给数据补0
之后用模2除法，将补0后的数据与生成多项式的比特串相除

$$\begin{array}{r} 110101 \\ 1101 \overline{) 101001000} \\ \underline{1101} \\ 00000000 \end{array}$$

只要比特数量足够就上1，不用管够不够减
因为就根本不是减法
异或运算，不同就是1，相同就是0





差错检测

循环冗余码

循环冗余码的使用步骤如下：

- 1.收发双方约定好一个**生成多项式 $G(x)$**
- 2.发送方根据要发送的数据和生成多项式计算出**差错检测码(冗余码)**，将其添加到待传输数据的后面一起传输
- 3.接收方通过生成多项式来计算收到的数据是否产生了**误码**

例：假设要发送的数据为101001，生成多项式为 $G(x)=x^3 + x^2 + 1$ ，如何计算出要发送的数据？

$$G(x)=x^3 + x^2 + 1 = \underset{\triangle}{1} \times \boxed{x^3} + \underset{\triangle}{1} \times x^2 + \underset{\triangle}{0} \times x^1 + \underset{\triangle}{1} \times x^0 \rightarrow \text{生成多项式构成的比特串为 } 1101$$

按照多项式的最高有效次项给数据补0
之后用模2除法，将补0后的数据与生成多项式的比特串相除

$$\begin{array}{r} 110101 \\ 1101 \overline{) 101001001} \end{array}$$

最后把剩下的余数加到被除数即可得到发送出去的数据



差错检测

循环冗余码

循环冗余码的使用步骤如下：

- 1.收发双方约定好一个**生成多项式 $G(x)$**
- 2.发送方根据要发送的数据和生成多项式计算出**差错检测码(冗余码)**，将其添加到待传输数据的后面一起传输
- 3.接收方通过生成多项式来计算收到的数据是否产生了误码

例：假设要发送的数据为101001，生成多项式为 $G(x)=x^3 + x^2 + 1$ ，如何计算出要发送的数据？

$$G(x)=x^3 + x^2 + 1 = \underset{\triangle}{1} \times \boxed{x^3} + \underset{\triangle}{1} \times x^2 + \underset{\triangle}{0} \times x^1 + \underset{\triangle}{1} \times x^0 \rightarrow \text{生成多项式构成的比特串为 } 1101$$

按照多项式的最高有效次项给数据补0
之后用模2除法，将补0后的数据与生成多项式的比特串相除

发送出去的数据：1 0 1 0 0 1 0 0 1

- 计算要发出的数据步骤：
- 1.根据生成多项式拿到除数的比特串(这个可能题目直接给)
 - 2.根据生成多项式最高次项给原始数据后面添0
 - 3.进行模2除法，除到最终的余数
 - 4.余数加回初始添0后的数据，即为要发出的数据

最后把剩下的余数加到被除数即可得到发送出去的数据



差错检测

循环冗余码

循环冗余码的使用步骤如下：

- 1.收发双方约定好一个**生成多项式G(x)**
- 2.发送方根据要发送的数据和生成多项式计算出**差错检测码(冗余码)**，将其添加到待传输数据的后面一起传输
- 3.接收方通过生成多项式来计算收到的数据是否产生了**误码**

生成多项式构成的比特串为**1101**

发送出去的数据：1 0 1 0 0 1 0 0 1 → 1 0 1 0 **1** 1 0 0 1 接收到的数据
传输过程中出现误码

那么接收方如何验证消息正确与否呢？

$$\begin{array}{r}
 1\ 1\ 0\ 1\ 1\ 0 \\
 1101 \overline{) 1\ 0\ 1\ 0\ \color{red}1\ 1\ 0\ 0\ 1} \\
 \underline{+ 1\ 1\ 0\ 1} \\
 1\ 1\ 1\ 1 \\
 \underline{+ 1\ 1\ 0\ 1} \\
 1\ 0\ 1\ 0 \\
 \underline{+ 1\ 1\ 0\ 1} \\
 1\ 1\ 1\ 0 \\
 \underline{+ 1\ 1\ 0\ 1} \\
 1\ 1\ 1
 \end{array}$$

用接收到的数据与约定的生成多项式比特串进行模2除法

如果到最后发现除不尽，那么就是传输过程中出现了错误



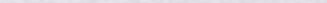
循环冗余码

1.收发双方约定好一个生成多项式 $G(x)$

2.发送方根据要发送的数据和生成多项式计算出差错检测码(冗余码), 将其添加到待传输数据的后面一起传输

3.接收方通过生成多项式来计算收到的数据是否产生了误码

生成多项式构成的比特串为1101

发送出去的数据: 1 0 1 0 0 1 0 0 1  1 0 1 0 1 1 0 0 1 接收到的数据

那么接收方如何验证消息正确与否呢?

Diagram illustrating a 4-bit ripple-carry adder circuit. The circuit consists of four full-adder blocks connected in series. The inputs and outputs are as follows:

- Block 1:** Inputs 1101 and 1101, Carry-in 0. Outputs: Sum 1001, Carry-out 1.
- Block 2:** Inputs 1101 and 1101, Carry-in 1. Outputs: Sum 1001, Carry-out 1.
- Block 3:** Inputs 1010 and 1101, Carry-in 1. Outputs: Sum 1101, Carry-out 1.
- Block 4:** Inputs 1110 and 1101, Carry-in 1. Outputs: Sum 111, Carry-out 1.

The final carry-out is 1.

用接收到的数据与约定的生成多项式比特串进行模2除法

如果到最后发现除不尽，
那么就是传输过程中出现了错误
用正确的数据检验一下





差错检测

循环冗余码

循环冗余码的使用步骤如下：

- 1.收发双方约定好一个**生成多项式 $G(x)$**
- 2.发送方根据要发送的数据和生成多项式计算出差错检测码(冗余码)，将其添加到待传输数据的后面一起传输
- 3.接收方通过生成多项式来计算收到的数据是否产生了误码

计算要发出的数据步骤：

- 1.根据生成多项式拿到除数的比特串(这个可能题目直接给)
- 2.根据生成多项式最高次项给原始数据后面添0
- 3.进行模2除法，除到最终的余数
- 4.余数加回初始添0后的数据，即为要发出的数据

收到以后验证数据正确的步骤：

- 1.拿到约定好的除数的比特串(这个可能题目直接给)
- 2.对收到的数据进行模2除法
- 3.如果余数是0，那就没问题，上交；如果余数非0，那就证明数据出错

注意：

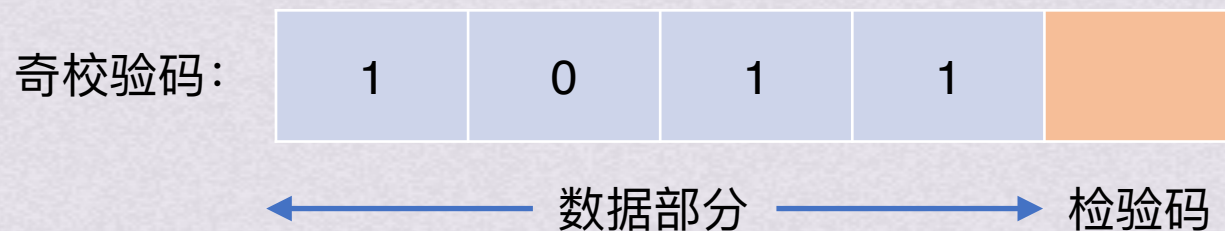
- 1.生成多项式必须包含最低次项(最后一个一定是1)
- 2.带 r 个检验位的多项式编码可以检测到所有长度 $\leq r$ 的错误
- 3.CRC有纠错能力，但是数据链路层只用到了检测错误的功能，出错了直接丢弃



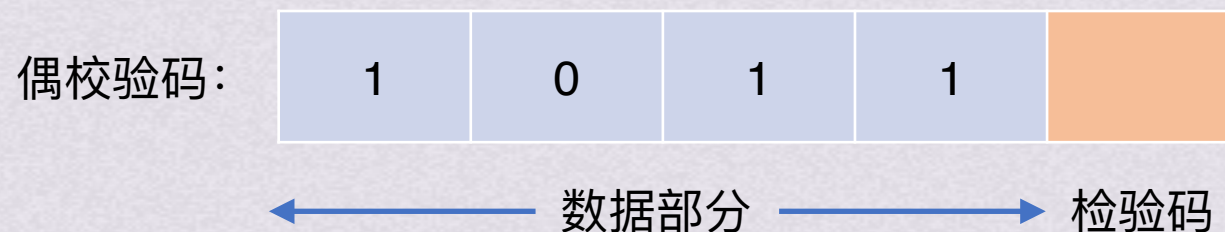
差错检测

奇偶校验码

检验整个数据部分，通过调整检验位的取值让整个检验码中的1为奇数(奇校验码)或偶数(偶校验码)



有3个1，要让1的个数是奇数，所以检验码位置填入0

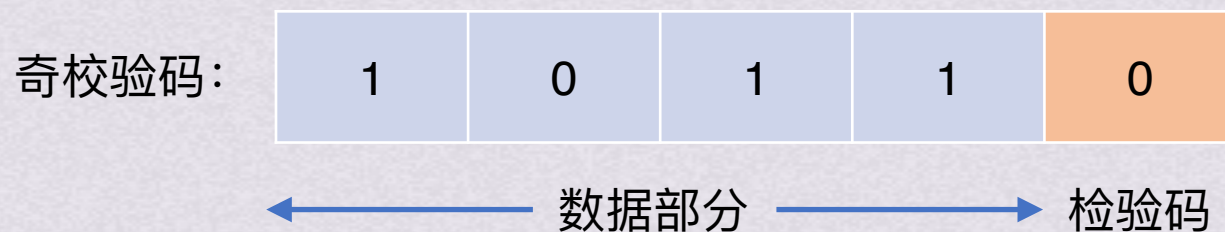




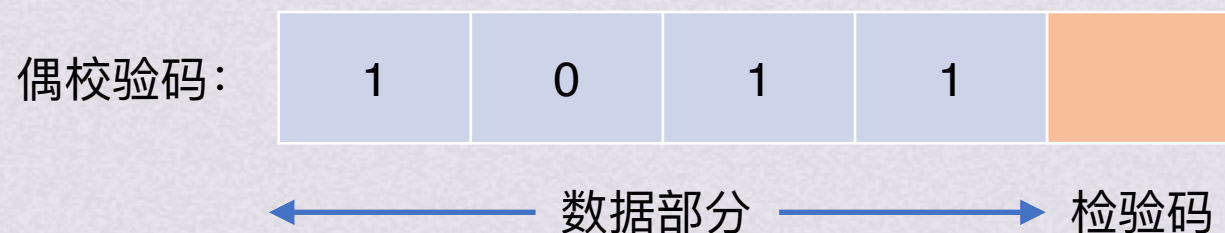
差错检测

奇偶校验码

检验整个数据部分，通过调整检验位的取值让整个检验码中的1为奇数(奇校验码)或偶数(偶校验码)



有3个1，要让1的个数是奇数，所以检验码位置填入0



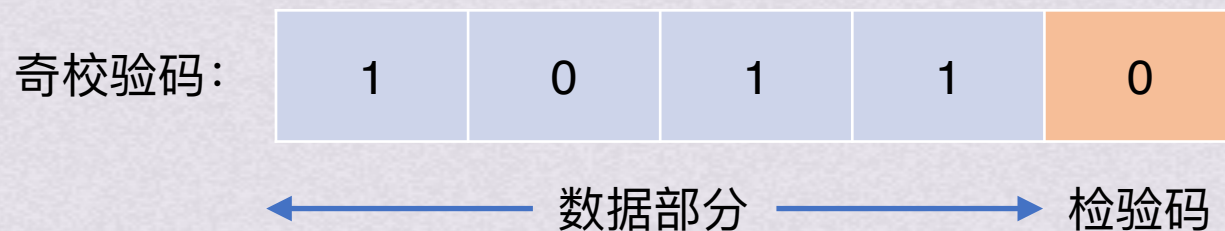
有3个1，要让1的个数是偶数，所以检验码位置填入1



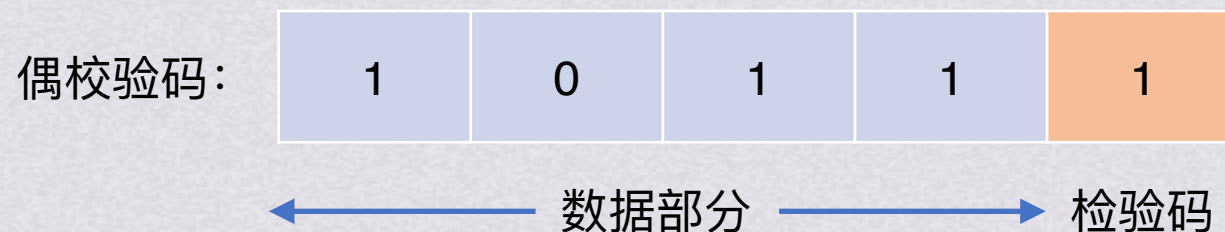
差错检测

奇偶校验码

检验整个数据部分，通过调整检验位的取值让整个检验码中的1为奇数(奇校验码)或偶数(偶校验码)



有3个1，要让1的个数是奇数，所以检验码位置填入0



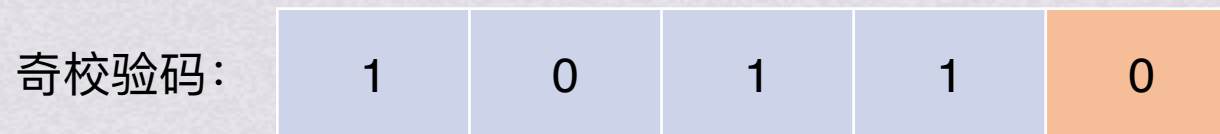
有3个1，要让1的个数是偶数，所以检验码位置填入1



差错检测

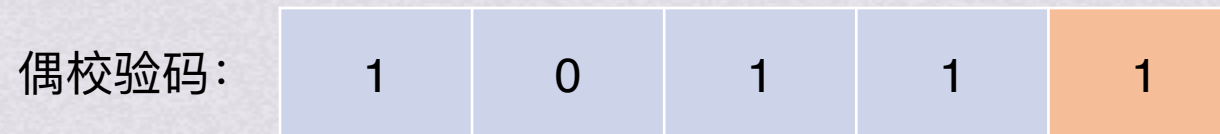
奇偶校验码

检验整个数据部分，通过调整检验位的取值让整个检验码中的1为奇数(奇校验码)或偶数(偶校验码)



← 数据部分 → 检验码

有3个1，要让1的个数是奇数，所以检验码位置填入0



← 数据部分 → 检验码

有3个1，要让1的个数是偶数，所以检验码位置填入1

奇偶校验码只能检测出奇数个比特的错误，一旦出现偶数比特个错误就无法检测出来



差错检测

海明码

是最常见的纠错编码，通过在数据中加入几位校验码来检测错误，并且有一定的纠正错误能力

为了检测或纠正错误，我们需要在数据中混入校验码，需要混入的校验码的比特数量使用公式计算

$$2^p - 1 \geq p + m$$

其中p是校验码的比特数，m是数据的比特数

以要发送4比特数据**1010**为例，m=4,则p≥3

校验码插在2的次方的位置，也就是1,2,4位置

位序	1	2	3	4	5	6	7
数据			1		0	1	0
	检验位	检验位	数据位	检验位	数据位	数据位	数据位

接下来我们计算一下检验位需要填入什么

海明码的原理是把数据拆散以后用好几组奇偶校验来确定有没有出问题
拆开数据的依据就是它的位序的二进制形式，所以我们先把位序换成二进制



差错检测

海明码

是最常见的纠错编码，通过在数据中加入几位校验码来检测错误，并且有一定的纠正错误能力

为了检测或纠正错误，我们需要在数据中混入校验码，需要混入的校验码的比特数量使用公式计算

$$2^p - 1 \geq p + m$$

其中p是校验码的比特数，m是数据的比特数

以要发送4比特数据 **1010** 为例，m=4,则p≥3

校验码插在2的次方的位置，也就是1,2,4位置

位序	1	2	3	4	5	6	7
位序(二进制)	001	010	011	100	101	110	111
数据			1		0	1	0
	检验位	检验位	数据位	检验位	数据位	数据位	数据位

接下来我们计算一下检验位需要填入什么

海明码的原理是把数据拆散以后用好几组奇偶校验来确定有没有出问题
拆开数据的依据就是它的位序的二进制形式，所以我们先把位序换成二进制

差错检测

海明码

是最常见的纠错编码，通过在数据中加入几位校验码来检测错误，并且有一定的纠正错误能力

位序	1	2	3	4	5	6	7
位序(二进制)	001	010	011	100	101	110	111
数据	p1	p2	1	p3	0	1	0
	检验位	检验位	数据位	检验位	数据位	数据位	数据位

接下来我们计算一下检验位需要填入什么

海明码的原理是把数据拆散以后用好几组奇偶校验来确定有没有出问题
拆开数据的依据就是它的位序的二进制形式，所以我们先把位序换成二进制
把最低比特是1的、中间比特是1的、最高比特是1的分别分成3组进行奇偶校验（这里用奇校验为例）

最低比特为1的数据是位序1,3,5,7的数据
奇校验1的个数为奇数，所以p1=0

p1	1	0	0
----	---	---	---



差错检测

海明码

是最常见的纠错编码，通过在数据中加入几位校验码来检测错误，并且有一定的纠正错误能力

位序	1	2	3	4	5	6	7
位序(二进制)	001	010	011	100	101	110	111
数据	p1	p2	1	p3	0	1	0
	检验位	检验位	数据位	检验位	数据位	数据位	数据位

接下来我们计算一下检验位需要填入什么

海明码的原理是把数据拆散以后用好几组奇偶校验来确定有没有出问题

拆开数据的依据就是它的位序的二进制形式，所以我们先把位序换成二进制

把最低比特是1的、中间比特是1的、最高比特是1的分别分成3组进行奇偶校验（这里用奇校验为例）

最低比特为1的数据是位序1,3,5,7的数据

p1=0	1	0	0
p2	1	1	0

奇校验1的个数为奇数，所以p1=0

中间比特为1的数据是位序2,3,6,7的数据

奇校验1的个数为奇数，所以p2=1



差错检测

海明码

是最常见的纠错编码，通过在数据中加入几位校验码来检测错误，并且有一定的纠正错误能力

位序	1	2	3	4	5	6	7
位序(二进制)	001	010	011	100	101	110	111
数据	p1	p2	1	p3	0	1	0
	检验位	检验位	数据位	检验位	数据位	数据位	数据位

接下来我们计算一下检验位需要填入什么

海明码的原理是把数据拆散以后用好几组奇偶校验来确定有没有出问题

拆开数据的依据就是它的位序的二进制形式，所以我们先把位序换成二进制

把最低比特是1的、中间比特是1的、最高比特是1的分别分成3组进行奇偶校验（这里用奇校验为例）

最低比特为1的数据是位序1,3,5,7的数据

p1=0	1	0	0
------	---	---	---

奇校验1的个数为奇数，所以p1=0

中间比特为1的数据是位序2,3,6,7的数据

p2=1	1	1	0
------	---	---	---

奇校验1的个数为奇数，所以p2=1

最高比特为1的数据是位序4,5,6,7的数据

p3	0	1	0
----	---	---	---

奇校验1的个数为奇数，所以p3=0



差错检测

海明码

是最常见的纠错编码，通过在数据中加入几位校验码来检测错误，并且有一定的纠正错误能力

位序	1	2	3	4	5	6	7
位序(二进制)	001	010	011	100	101	110	111
数据	p1	p2	1	p3	0	1	0
	检验位	检验位	数据位	检验位	数据位	数据位	数据位

接下来我们计算一下检验位需要填入什么

海明码的原理是把数据拆散以后用好几组奇偶校验来确定有没有出问题

拆开数据的依据就是它的位序的二进制形式，所以我们先把位序换成二进制

把最低比特是1的、中间比特是1的、最高比特是1的分别分成3组进行奇偶校验（这里用奇校验为例）

最低比特为1的数据是位序1,3,5,7的数据

p1=0	1	0	0
p2=1	1	1	0
p3=0	0	1	0

奇校验1的个数为奇数，所以p1=0

中间比特为1的数据是位序2,3,6,7的数据

奇校验1的个数为奇数，所以p2=1

最高比特为1的数据是位序4,5,6,7的数据

奇校验1的个数为奇数，所以p3=0



差错检测

海明码

是最常见的纠错编码，通过在数据中加入几位校验码来检测错误，并且有一定的纠正错误能力

位序	1	2	3	4	5	6	7
位序(二进制)	001	010	011	100	101	110	111
数据	0	1	1	0	0	1	0
	检验位	检验位	数据位	检验位	数据位	数据位	数据位

接下来我们计算一下检验位需要填入什么

海明码的原理是把数据拆散以后用好几组奇偶校验来确定有没有出问题

拆开数据的依据就是它的位序的二进制形式，所以我们先把位序换成二进制

把最低比特是1的、中间比特是1的、最高比特是1的分别分成3组进行奇偶校验（这里用奇校验为例）

最低比特为1的数据是位序1,3,5,7的数据	p1=0	1	0	0	奇校验1的个数为奇数，所以p1=0
中间比特为1的数据是位序2,3,6,7的数据	p2=1	1	1	0	奇校验1的个数为奇数，所以p2=1
最高比特为1的数据是位序4,5,6,7的数据	p3=0	0	1	0	奇校验1的个数为奇数，所以p3=0



差错检测

海明码

是最常见的纠错编码，通过在数据中加入几位校验码来检测错误，并且有一定的纠正错误能力

为了检测或纠正错误，我们需要在数据中混入校验码，需要混入的校验码的比特数量使用公式计算

$$2^p - 1 \geq p + m$$

其中p是校验码的比特数，m是数据的比特数

以要发送4比特数据 **1010** 为例，m=4,则p≥3

校验码插在2的次方的位置，也就是1,2,4位置

位序	1	2	3	4	5	6	7
位序(二进制)	001	010	011	100	101	110	111
数据	0	1	1	0	0	1	0
	检验位	检验位	数据位	检验位	数据位	数据位	数据位

于是我们就得到了二进制数据1010的奇校验海明码为0110010



差错检测

海明码

是最常见的纠错编码，通过在数据中加入几位校验码来检测错误，并且有一定的纠正错误能力

为了检测或纠正错误，我们需要在数据中混入校验码，需要混入的校验码的比特数量使用公式计算

$$2^p - 1 \geq p + m$$

其中p是校验码的比特数，m是数据的比特数

以要发送4比特数据 **1010** 为例，m=4,则p≥3

校验码插在2的次方的位置，也就是1,2,4位置

位序	1	2	3	4	5	6	7
位序(二进制)	001	010	011	100	101	110	111
数据	0	1	1	0	0	1	0

假设发送过程中出现误码

收到的数据	0	1	1	0	1	1	0
-------	---	---	---	---	---	---	---



差错检测

海明码

是最常见的纠错编码，通过在数据中加入几位校验码来检测错误，并且有一定的纠正错误能力

位序	1	2	3	4	5	6	7
位序(二进制)	001	010	011	100	101	110	111
数据	0	1	1	0	0	1	0

假设发送过程中出现误码

位序	1	2	3	4	5	6	7
位序(二进制)	001	010	011	100	101	110	111
收到的数据	0	1	1	0	1	1	0

和刚才一样，按照不同位置比特=1分类，把数据填入对应的位置

p1=0	1	1	0	本组1的个数为偶数，出错
p2=1	1	1	0	本组1的个数为奇数，没问题
p3=0	1	1	0	本组1的个数为偶数，出错



差错检测

海明码

是最常见的纠错编码，通过在数据中加入几位校验码来检测错误，并且有一定的纠正错误能力

位序	1	2	3	4	5	6	7
位序(二进制)	001	010	011	100	101	110	111
数据	0	1	1	0	0	1	0

假设发送过程中出现误码

位序	1	2	3	4	5	6	7
位序(二进制)	001	010	011	100	101	110	111
收到的数据	0	1	1	0	1	1	0

和刚才一样，按照不同位置比特=1分类，把数据填入对应的位置

p1=0	1	1	0
p2=1	1	1	0
p3=0	1	1	0

本组1的个数为偶数，出错 位序1,3,5,7中有错

本组1的个数为奇数，没问题 位序2,3,6,7没出错

本组1的个数为偶数，出错 位序4,5,6,7中有错



差错检测

海明码

是最常见的纠错编码，通过在数据中加入几位校验码来检测错误，并且有一定的纠正错误能力

位序	1	2	3	4	5	6	7
位序(二进制)	001	010	011	100	101	110	111
数据	0	1	1	0	0	1	0

假设发送过程中出现误码

位序	1	2	3	4	5	6	7
位序(二进制)	001	010	011	100	101	110	111
收到的数据	0	1	1	0	1	1	0

和刚才一样，按照不同位置比特=1分类，把数据填入对应的位置

p1=0	1	1	0
p2=1	1	1	0
p3=0	1	1	0

本组1的个数为偶数，出错

位序1,5中有错

本组1的个数为奇数，没问题

位序2,3,6,7没出错

本组1的个数为偶数，出错

位序4,5中有错

成功定位到错误在位序5的比特上，
反转比特就拿到了原始数据



差错检测

海明码

是最常见的纠错编码，通过在数据中加入几位校验码来检测错误，并且有一定的纠正错误能力

位序	1	2	3	4	5	6	7
位序(二进制)	001	010	011	100	101	110	111
数据	0	1	1	0	0	1	0

假设发送过程中出现误码

位序	1	2	3	4	5	6	7
位序(二进制)	001	010	011	100	101	110	111
收到的数据	0	1	1	0	1	1	0

和刚才一样，按照不同位置比特=1分类，把数据填入对应的位置

回到这一步骤，还有一个快捷定位错误的方法，就是对三行数据再次进行奇校验

然后把3bit的校验码 顺时针旋转90度，最上面的比特变成最右边的比特

1	p1=0	1	1	0
0	p2=1	1	1	0
1	p3=0	1	1	0



差错检测

海明码

是最常见的纠错编码，通过在数据中加入几位校验码来检测错误，并且有一定的纠正错误能力

位序	1	2	3	4	5	6	7
位序(二进制)	001	010	011	100	101	110	111
数据	0	1	1	0	0	1	0

假设发送过程中出现误码

位序	1	2	3	4	5	6	7
位序(二进制)	001	010	011	100	101	110	111
收到的数据	0	1	1	0	1	1	0

和刚才一样，按照不同位置比特=1分类，把数据填入对应的位置
回到这一步骤，还有一个快捷定位错误的方法，就是对三行数据再次进行奇校验
然后把3bit的校验码 顺时针旋转90度，最上面的比特变成最右边的比特

		1	
0	p1=0	1	1
	p2=1	1	1
1	p3=0	1	1



差错检测

海明码

是最常见的纠错编码，通过在数据中加入几位校验码来检测错误，并且有一定的纠正错误能力

位序	1	2	3	4	5	6	7
位序(二进制)	001	010	011	100	101	110	111
数据	0	1	1	0	0	1	0

假设发送过程中出现误码

位序	1	2	3	4	5	6	7
位序(二进制)	001	010	011	100	101	110	111
收到的数据	0	1	1	0	1	1	0

和刚才一样，按照不同位置比特=1分类，把数据填入对应的位置
回到这一步骤，还有一个快捷定位错误的方法，就是对三行数据再次进行奇校验
然后把3bit的校验码 顺时针旋转90度，最上面的比特变成最右边的比特

0 1

p1=0	1	1	0
p2=1	1	1	0
p3=0	1	1	0



差错检测

海明码

是最常见的纠错编码，通过在数据中加入几位校验码来检测错误，并且有一定的纠正错误能力

位序	1	2	3	4	5	6	7
位序(二进制)	001	010	011	100	101	110	111
数据	0	1	1	0	0	1	0

假设发送过程中出现误码

位序	1	2	3	4	5	6	7
位序(二进制)	001	010	011	100	101	110	111
收到的数据	0	1	1	0	1	1	0

和刚才一样，按照不同位置比特=1分类，把数据填入对应的位置

回到这一步骤，还有一个快捷定位错误的方法，就是对三行数据再次进行奇校验

然后把3bit的校验码 顺时针旋转90度，最上面的比特变成最右边的比特

1 0 1 → 错误在位序5

p1=0	1	1	0
p2=1	1	1	0
p3=0	1	1	0



差错检测

海明码

是最常见的纠错编码，通过在数据中加入几位校验码来检测错误，并且有一定的纠正错误能力

为了检测或纠正错误，我们需要在数据中混入校验码，需要混入的校验码的比特数量使用公式计算

$$2^p - 1 \geq p + m$$

其中p是校验码的比特数，m是数据的比特数

以要发送4比特数据 **1010** 为例，m=4,则p≥3

还要学习另一个公式 $L-1=D+C(D \geq C)$ ，其中L为海明距，D为检测错误比特的个数，C为纠错比特的个数

海明距越大，纠错和检测错误的能力越强，上文学习的是海明距为3的海明码，所以可以检测2bit错误或者修正1bit的错误，这一部分内容了解即可